

Proposal: переработка процесса выработки и оформления спецификации

Статус: черновик для вычитки **Дата:** 2026-06-07 **Область:** агенты планирования в `roles/devbox/files/dot_claude/` (TPM, domain-expert, domain-modeller) и формат их вывода **Не входит в область:** оркестрация полного цикла разработки и гейты (`/full-cycle`) — отдельная тема

1. Зачем это всё (проблема)

Текущий процесс выработки спецификации страдает от трёх связанных болезней:

1. **Монолит.** Спек — один большой файл. Неудобно читать ни человеку, ни LLM. То же с `domain_analysis.md` и `domain_model.md` — длинные простыни.
2. **Скрытый waterfall.** Пайплайн режет работу горизонтальными слоями: сначала вся спека → потом весь домен → потом весь план → потом код. Это Big Design Up Front. Для сложных (по Synefin) доменов это вредно: первый же POC обесценит половину допущений, которые мы уже «довели до конца».
3. **Тяжесть и ручной труд.** Человек руками дёргает `/domain-analysis`, `/plan`, `/api-design`. Каждая стадия — обязательная и рождает отдельную простыню. Много ручных операций, мало параллельности.

Дополнительно — дублирование: каждый агент пишет `X.md` + `X_output.json`, две заметки об одном и том же. Это уже существующая графомания.

Важно: многое из нужного уже есть как кирпичи (append-only лог решений, progress-spine как DAG, агент observability, skip-условие для модельера). Проблема не в отсутствии кирпичей, а в их сборке в waterfall-конфигурацию.

2. Цель и не-цели

Цель: переработать, как существующие 2-3 агента (TPM, domain-expert, domain-modeller) продуцируют свои данные, валидируют их и договариваются, и сменить форму вывода с монолита на связанный набор заметок (vault). Получить на выходе спецификацию + план развития + доменную модель как единый граф заметок.

Явные не-цели: - Не плодить новые сущности и агентов ради красоты. Бритва Оккама. - Не заниматься бюрократией и графоманией. Заметка существует, только если несёт **решение, ограничение или термин**. - Не строить архитектуру ради архитектуры.

Главные ценности: максимальная скорость и параллельность движения вперёд при минимуме ручных операций.

3. Ключевая идея: слайсы вместо слоёв

Критика waterfall не означает «без фаз». Фазы будут — `foundation` → `POC` → `MVP` → `roadmap`. Разница в **направлении нарезки работы**:

- **Сейчас (waterfall):** горизонтальные слои. Вся спека → весь домен → весь план → код.
- **Предлагается:** вертикальные слайсы. Каждая фаза — сквозной кусок «от спекы до работающего кода», и спека пишется **ровно настолько, насколько нужно текущей фазе**.

Это известная практика **walking skeleton / tracer bullet**. Она даёт чёткий критерий достаточности спекы:

- **foundation** — тончайший сквозной скелет, на котором решены «детские болезни»: логирование, конфиг, обработка ошибок, CI, **харнесс телеметрии**, тестовый каркас (paved road).
- **POC** — проверка «оно вообще работает или нет».
- **MVP** — определённый список того, что нужно.
- **roadmap** — то, что осознанно отложено.

Анти-waterfall механизм — это не отсутствие фаз, а **право процесса вернуться в спеку после РОС**, узнав «работает / не работает», и переписать `to-be MVP` с учётом узнанного. Цикл повторно входит в спеку (re-entry). Сейчас этого нет — движение только вперёд по гейтам.

4. Что такое спецификация (содержание)

Спека отвечает: что строим, зачем, для кого, как выглядит успех, что в скоупе и вне его, какие ограничения и риски, какова форма проблемной области. Спека — это **не** план работ (это plan) и **не** код.

Ядро содержания:

1. **Intent** — зачем, кому больно, какой end-state (состояния, не задачи).
2. **Actors & jobs** — кто пользуется, какие jobs-to-be-done.
3. **Capabilities** — что система делает, как исходы (Given-When-Then).
4. **Quality attributes / NFR** — производительность, безопасность, масштаб, надёжность — с числами и таргетами по фазам.
5. **Domain** — ubiquitous language, сущности, инварианты, события (линкуется, ленивый).
6. **Constraints & assumptions** — допущения помечены под валидацию.
7. **Observability** — какие метрики, success + guardrail, по фазам. Аудит/телеметрия/продуктовые метрики из коробки.
8. **Phases** — `foundation/POC/MVP/roadmap`, у каждой свой DoD и таргеты метрик.
9. **Decisions (ADR)** — почему так, append-only.
10. **Out-of-scope / no-gos** — явно.
11. **Risks & open questions** — живой список.

Чего в спеке **нет**: разбивки на задачи, классов/кода, исчерпывающего перебора edge-кейсов (только инварианты + кейсы текущей фазы).

5. Greenfield vs Brownfield

Это не нюанс, а **разное направление авторства при одной целевой структуре vault**.

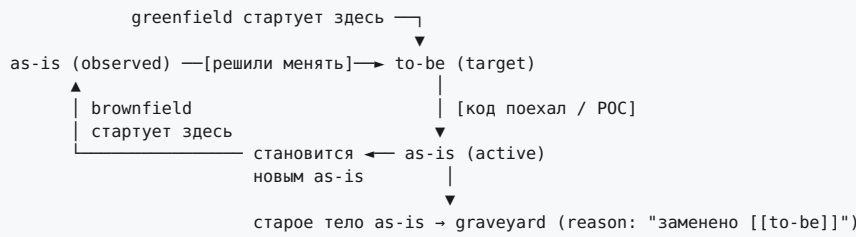
- **Greenfield** — спека генеративна, авторство сверху-вниз. Начинаем с intent, спека ведёт, код следует. Домен изобретается. Риск — фантазия/BDUF, лечится walking-skeleton + re-entry.
- **Brownfield** — спека реконструктивна, авторство снизу-вверх, затем дельта. Система уже существует, «настоящая спека» зашита в код, схему БД и поведение. Это археология: извлекаем as-is из кода/телеметрии → документируем baseline → изменение = дельта `as-is → to-be`.

В brownfield источник истины — **код + прод-телеметрия**, а не веб-ресёрч. И тот же принцип лени: реконструируем **только тот слайс, который трогаем**, а не всю систему авансом.

Greenfield/brownfield — это режимы существующих агентов, а не новые агенты. Brownfield-режим = read-only анализ кода (умеет `architect`) + обратный `domain-modeller`.

6. Жизненный цикл заметки (снимает дихотомию as-is/to-be)

`as-is` и `to-be` — не два каталога, а статусы на пути одной заметки. Это складывается в append-only механизм:



- Один lifecycle унифицирует greenfield (стартует с `to-be`), brownfield (с `as-is`) и append-only историю.
- `as-is` не хоронится в момент решения — он остаётся истинной документацией текущей реальности, пока код не поехал. До этого `as-is` и `to-be` сосуществуют.
- **Living documentation чинит себя сама**: сегодняшний `to-be` — завтрашний `as-is`. Документация `as-is` накапливается как побочный продукт тронутых слайсов, без титанических усилий.

7. Формат вывода: vault atlas/

Спека хранится в каталоге-vault в корне проекта (формат в стиле Obsidian — заметки со взаимными ссылками, индексные файлы). Имя — `atlas/` (атлас = собрание карт, рифмуется с MOC, maps of content; не коллизит с `docs/` и с `ansible-vault/`).

7.1. Раскладка

```

atlas/
index.md           # корневой MOC: charter + активные специ + указатель «текущая фаза»
charter.md         # north-star (raw + interpreted intent), raw – immutable
domain/           # ОБЩИЙ на проект (шарится между специ)
  index.md         # domain landscape MOC: контексты + владельцы
  contexts/
    <context>/
      index.md
      overview.md  # назначение, subdomain-type (core|supporting|generic), owner/team
      glossary.md  # ubiquitous language ЛОКАЛЬНО для контекста
      <aggregate>.md
      events.md
  decisions/       # решения уровня domain
  specs/
    <spec-name>/
      index.md     # MOC спеки: проекция на контексты, текущая фаза, открытые риски
      problem.md   actors.md
      capabilities/<cap>.md
      nfr.md constraints.md assumptions.md observability.md
      phases/{foundation,poc,mvp,roadmap}.md
      challenges/<challenge>.md # заметки-критика domain-expert
      decisions/   # решения уровня spec
  decisions/       # решения уровня project (кросс-каттинг)
  _archive/        # приостановленные кластеры (warm, перечитывается при возобновлении)
  _graveyard/      # мёртвые тела заметок (cold, только трассировка «почему»)
  
```

Домен — на уровне проекта, спека ссылается в него. Спец ⊥ Domain: спец — единица изменения/намерения; domain — единица смысла/владения. Одна спека может покрывать несколько доменов.

7.2. Типы заметок

type	назначение	кто пишет
charter	задумка, north-star	steward
moc	индекс/точка входа	steward (частично авто-генерит линтер)
problem	боль, контекст	steward
actor	персона + jobs-to-be-done	steward
capability	что система делает (исход + GWT)	steward
nfr	quality attribute с числами/таргетами по фазам	steward + observability
constraint	тех/орг/регуляторное ограничение	steward
assumption	допущение под валидацию	steward, валидирует expert
domain-model	агрегат/контекст, инварианты, события	domain-modeller
glossary	ubiquitous language (локально для контекста)	domain-modeller
challenge	критика к заметке	domain-expert
observability	метрики: что меряем, success + guardrail	observability
phase	фаза + её DoD + таргеты метрик	steward
decision	ADR	steward

7.3. Frontmatter (общий каркас)

```
---
id: cap-guest-checkout           # стабильный slug = адрес для ссылок
title: Guest checkout
type: capability
status: to-be                   # draft|as-is|to-be|active|paused|superseded|rejected
phase: poc                     # foundation|poc|mvp|roadmap
owner: spec-steward
created: 2026-06-07
updated: 2026-06-07
traces_to: "[[goal-reduce-cart-abandonment]]"
supersedes: "[[cap-guest-checkout-v0]]"
superseded_by: null
reason: null
tags: [checkout, payments]
---
```

Frontmatter = машинный контракт. Это убирает дублирующий `*_output.json`: структурные метаданные живут во frontmatter, человекочитаемо. Сводный машинный индекс при необходимости генерит линтер в один `index.json`.

7.4. Обязательные поля по статусу (проверяет линтер)

статус / тип	сверх базовых полей
любой	id, title, type, status, owner, created, updated
capability	traces_to (цепочка вверх до charter)
as-is	source — где наблюдали (путь в коде / телеметрия / схема)
to-be	traces_to + acceptance (Given-When-Then в теле)
superseded	superseded_by + reason → тело в _graveyard/
paused	reason + resume_entry → кластер в _archive/
rejected	reason
assumption	validation (open validated invalidated) + risk_if_wrong
challenge	targets [[note]] + severity + resolution (open accepted dismissed)
decision	ADR-статус (proposed accepted deprecated superseded) + scope + context/decision/consequences

7.5. Archive vs Graveyard

Различаются **намерением вернуться**, а не свежестью:

	Archive	Graveyard
Семантика	приостановлено, возможно валидно	мертво / заменено / удалено
Триггер	active → paused (пауза инициативы)	active → superseded/rejected
Гранулярность	обычно кластер заметок	обычно одна заметка
Чтение LLM	перечитывается при возобновлении (warm)	не читается, кроме трассировки «почему» (cold)
Обратный переход	archive → active возможен	назад нет
Обязательные поля	paused_reason , resume_entry	superseded_by + reason

8. Мультидомен

Большинство систем мультидоменные. **Домены — first-class** (domain/contexts/<context>/), у каждого свой владелец/team (Conway) и subdomain-type (core/supporting/generic).

Важный сдвиг: **гlossарий локален для контекста, не глобален**. Одно слово (Order , Customer , Account) значит разное в разных контекстах — и эта неоднозначность на границе есть топ-источник багов. Поэтому UL живёт в contexts/<ctx>/glossary.md .

Кросс-доменное взаимодействие — это содержание спеки, а не отдельный аппарат. Сознательно **не** вводим формальные seams/ -заметки и context-map-машинерию на этом этапе (архитектура ради архитектуры, бритва Оккама). Если capability трогает несколько контекстов — взаимодействие описывается прямо в ней. В frontmatter capability:

```
contexts: ["[[ctx-sales]]", "[[ctx-billing]]"]
```

Идею hub-заметок для стыков фиксируем как возможное будущее, но сначала обкатываем каркас на реальности.

9. Решения (decisions) и проблема «10000 файлов»

Append-only лог не должен превратиться в папку на 10000 файлов. Ответ — Оккам, в четыре приёма:

- 1. **Большинство «решений» вообще не отдельные файлы**. «Почему» уже захвачено жизненным циклом: при супер-сиде причина лежит в reason / superseded_by затронутой заметки. Отдельный ADR нужен **только когда у решения нет одной заметки-дома** — кросс-каттинг вроде «берём Postgres», «идём в event-driven». Это убирает ~90% мнимой кардинальности.
- 2. **Колокация по scope, а не общая свалка**. Scope определяет место (и заодно закрывает старый спор «единый лог vs spec-local»): - project → atlas/decisions/ - domain → atlas/domain/<ctx>/decisions/ - spec → atlas/specs/<name>/decisions/

Файлы распределяются по дереву и читаются в контексте — ни одна папка не взрывается. В frontmatter решения: scope + affects: [[...]] .

- 1. **Холодная архивация**. deprecated / superseded решения уезжают в _graveyard/ тем же механизмом, что и заметки. История не теряется, но уходит из рабочего набора.
- 2. **Высокая планка**. ADR — для значимого, трудно-обратимого, кросс-каттинг. Рутинный выбор — строка в теле релевантной заметки, а не файл. 10000 ADR — симптом графомании, а не масштаба.

Ключевое: ограничение — не «диск не вместит», а «рабочий набор LLM не должен грузить 10000 файлов». А он и не грузит: читается только текущий scope со status: active . Рабочий набор ограничен фильтром scope + status, а не суммарной историей.

10. Роли трёх агентов: продукция / валидация / договорённость

Тот же набор людей, сменённые режимы. Никаких новых агентов.

	Продуцируют	Валидируют	Договариваются
TPM	пишет спину atlas (charter, problem, actors, capabilities, phases, observability) — мульти-заметки вместо spec.md + spec_output.json	—	владелец графа: интегрирует, держит трассировку до charter
domain-expert	из автора domain_analysis.md → в критика: короткие challenge-заметки + статус допущений	челлендж допущений и опасных инвариантов	спор TPM ↔ expert разрешается в decision с rationale
domain-modeller	заметки domain/contexts/<ctx>/, лениво по слайсу	self-check арператов	—

Сквозные механизмы (а не новые сущности):

- **Продукция:** frontmatter = машинный контракт → убирает дубль *_output.json. Одна заметка вместо двух файлов.
- **Валидация:** domain-expert (смысловая) + vault-lint (структурная: битые ссылки, сироты, обязательные поля по статусу, трассировка до charter).
- **Договорённость = append-only история:** decision со scope + reason, supersession с причиной, статус-lifecycle. Любое расхождение между агентами фиксируется как решение «для истории».

11. Спеc-steward (эволюция TPM, а не новый агент)

Чтобы убрать ручное дёрганье слэш-команд: **TPM эволюционирует** в роль, которая пишет спеку и точно дёргает expert/modeller/observability как инструменты.

Внешний контур: фазы со сквозными слайсами

foundation → POC → MVP → roadmap
каждая фаза = тонкий вертикальный слайс:
спека(слайс) → имплементация → УЗНАЛИ → re-entry в спеку

Внутренний контур: авторский burn-down

Стюард держит **фронтитр неизвестностей** (open_questions + unvalidated_assumptions) и гасит их, точно вызывая специалистов:

0. SEED (определить режим входа)
трогаем существующий код? — да → reality-led: реконструкция as-is слайса
— нет → intent-led: набросок intent + capabilities (to-be)

loop (пока не выполнен Slice-DoD):

1. выбрать неизвестность с наибольшим рычагом
2. РОУТИНГ (можно несколько ПАРАЛЛЕЛЬНО — note-level, мало конфликтов):
 - спорное допущение → domain-expert (critic: challenge-заметка)
 - нужен формальный домен → domain-modeller (только текущий контекст)
 - метрики/телеметрия фазы → observability
 - нужна реальность кода → reality-led анализ (architect)
 - развилка без «правильного» → отложить в очередь чекпоинтов
3. интегрировать ответы в граф, обновить МОС
4. новые ответы могут породить новые неизвестности → назад в фронтитр
5. очередь чекпоинтов блокирует? → ОДИН батч-чекпоинт человеку (пачкой развилки)

до: Slice-DoD

Slice-DoD (анти-waterfall на уровне авторства)

Стюард отдаёт в имплементацию **не когда «спека готова»**, а когда решено ровно столько, чтобы безопасно построить тончайший следующий слайс:

- capabilities слайса имеют Given-When-Then;

- инварианты тронутого контекста сформулированы;
- не осталось **блокирующих** непровалидированных допущений (неблокирующие → в risks);
- метрики фазы определены;
- пройден батч-чекпоинт, если была реальная развилка.

Всё прочее — осознанно «не сейчас», уезжает в roadmap.

Что держит лёгкость

1. **Точечный диспатч, не стадии.** Специалист зовётся на конкретную заметку, а не как обязательная фаза-простыня.
2. **Батчинг чекпоинтов.** Человека дёргаем редко и пачкой развилочек.
3. **Параллельный фан-аут.** Независимые неизвестности уходят специалистам одновременно; стюард собирает. Низкий конфликт интеграции за счёт note-level гранулярности.

Защита от потери задумки

- **charter — якорная заметка.** `raw_intent` (дословные слова пользователя, immutable) + `interpreted_intent` (дистилляция, правится). Первый дешёвый чекпоинт: «верно понял?».
- **Трассируемость вверх до charter.** Линтер проверяет: активная заметка без цепочки до charter — либо score сгееер (выкинуть), либо charter неполный (дополнить).
- **Перечитывание на каждом re-entry.** North-star переутверждается каждую петлю.
- **Intent меняется только явно** — через ADR с причиной, не молча.

12. Валидация: `vault-lint`

Vault без проверки ссылок сгниёт (LLM плохо держит граф линков). Нужен `bin/vault-lint`:

- битые wikilinks;
- осиротевшие заметки (недостижимы из МОС);
- обязательные поля по статусу (см. 7.4);
- трассировка каждой активной capability вверх до charter;
- у `superseded` есть `superseded_by` + `reason`; у `paused` — `resume_entry`.

Впишется в существующую культуру валидаторов (`validate-claude`, `validate-pipeline-output`).

13. Принципы против графомании

- Заметка существует, только если её удаление теряет **решение, ограничение или термин**.
- Тело — минимальные шаблоны, без разделов «для галочки» (линтер ругается на пустые секции-заглушки).
- Frontmatter вместо дублирующего JSON.
- Высокая планка для ADR.
- Лень: моделируем/реконструируем только текущий слайс, не всю систему.

14. Открытые вопросы и TODO-ресёрчи

- **TODO (research): единица декомпозиции спекы.** Гранулярность capability vs user-story vs job vs use-case. У всех подходов свои ловушки (user stories тоже вводят в заблуждение). На входе — текущий TPM (pitch-формат, GWT), допиливаем по итогам.
- **TODO: spec-steward agent** — формальная спека агента-оркестратора, заменяющего ручные слэш-команды (эволюция TPM).
- **TODO: bin/vault-lint** — структурный валидатор vault.
- **Открыто:** hub-заметки для кросс-доменных стыков — отложено, сначала обкатать каркас.

- **Открыто:** приоритизация фронта стюардом — простая эвристика (блокеры > высокий риск-если-неверно > формализация > nice-to-know), без сложного скоринга.
 - **Открыто:** триггер re-entry после РОС — «всегда, но дешево» (короткая retro-заметка; нет расхождений — петля проходит мгновенно).
 - **Открыто:** foundation как спец-слайс с фиксированным DoD-шаблоном (paved road).
-

15. Что мы сознательно НЕ делаем

- Не вводим формальные seams/ и context-map-машинерию (кросс-доменное — в спеке).
 - Не плодим новых агентов (greenfield/brownfield и steward — режимы существующих).
 - Не делаем единый плоский лог решений (score + колокация + холодная архивация).
 - Не трогаем оркестрацию полного цикла и гейты — отдельная тема.
 - Не пишем спеку «до конца» перед кодом (Slice-DoD + re-entry).
-

Конец proposal. Документ — основа для обкатки на реальной спеке; уточняется по итогам.